

VIRTUAL MEMORY INTERNALS

1. Memory Operations
2. Process (Virtual) Address Space
3. Virtual Memory and Physical Memory
4. (Virtual) Memory Operations: Reservation vs Allocation
5. The Page Table
6. Conclusion and Takeaways

PITCH

- **demo:** number of page faults by an application & system-level
- **demo:** total value of RSS vs. total physical memory used

PROCESS VIRTUAL ADDRESS SPACE (OS VIEW)

```
struct zone *pvas_head;
struct zone {
    unsigned long start;
    unsigned long size;
    unsigned int permissions;
    struct zone *prev, *next;
};
```

PROCESS VIRTUAL ADDRESS PAGE (HARDWARE / PAGE TABLE VIEW)

```
struct page page_table[N];  
struct page {  
    bool valid;  
    unsigned int frame_index;  
    unsigned int permissions;  
    unsigned int flags;  
}
```

PROCESS VIRTUAL ADDRESS SPACE

- **demo:** `pmap` \$\$ how many zones, how many valid PTEs
- **diagram:** zones, page table, correspondence

MANAGING THE VIRTUAL ADDRESS SPACE

- zones: managed by OS
- page table: managed by hardware / MMU / MTU (Memory Management Unit, Memory Translation Unit)

VIRTUAL TO PHYSICAL ADDRESS TRANSLATION

- we only use virtual addresses
- done by MMU / MTU
- page address + offset -> translate page address to frame address -> add offset

VIRTUAL MEMORY RESERVATION

- only affects the list of zones view
- does not affect page table
- reserved explicitly by a memory allocation call
(`malloc()`, `mmap()`)
- **diagram:** reserve virtual memory

PHYSICAL MEMORY ALLOCATION AND MAPPING

- only affects the page table - validity bit is updated
- virtual memory areas not affected
- allocate at access - not explicit
- **diagram:** allocation of physical memory, mapping frames to pages, validation of PTEs

PAGE FAULT

- **demo:** show page fault metrics of processes
- a page fault is triggered when access is done to a page that's invalid or with the wrong permissions

PAGE FAULT FLOW

- **diagram** with flow
- memory access instruction
- aim to translate virtual address to physical address (MMU / MTU)
- if page table entry is invalid, trigger page fault
- run page fault handler
- check the zones view (OS)
- allocate or find physical page (frame)
- map frame to page: update page table entry

PAGE FAULT METRICS

- **demo:** reservation and allocation, see `vsz`, `rss`, `minflt`, `majflt` metrics

SEGMENTATION FAULT

- page fault types: minor page fault, major page fault, invalid page fault (causes seg fault), permissions page fault (generally causes segfault)
- **demo:** seg faults

PAGE FAULT SCENARIOS

- diagram: demand paging
- diagram: swapping

MEMORY SHARING

- **demo:** resident memory used by 10 processes started from:
 - the same dynamic executable
 - the same static executable
 - 10 different dynamic executables
 - 10 different static executables

MEMORY SHARING TYPES

- **diagram:** same physical memory, multiple address spaces accessing it
- **implicit:** executables, libraries
- **explicit:** `shm . . .` calls, require synchronization - part of App Interaction lectures

STORING THE PAGE TABLE

- page table in memory
- two issues:
 - double memory access: use TLB - a cache of the page table: **diagram**
 - a lot of space use multi-level paging: **diagram**

CONCLUSION AND TAKEAWAYS

- virtual memory gives the appearance of using more memory than actually being used
- physical memory is allocated at access
- page table stores state of the physical page
- invalid page may mean not part of the VAS (segfault at access), or not yet allocated / mapped (no segfault at access)

CONCLUSION AND TAKEAWAYS (2)

- allocation is triggered by access, and by signal sent by the MMU / MTU to the operating system (page fault handler)
- frames not used are swapped on disk (swap space)
- benefit of virtual memory is memory sharing
- downside of virtual memory is space allocated for page table (alleviated with multi-level paging) and double memory access (page table + actual access) (alleviated by TLB)